



TECNICATURA SUPERIOR EN

**Ciencia de Datos e
Inteligencia Artificial**

TÉCNICAS DE PROCESAMIENTO DEL HABLA

EJE N°1: Fundamentos y Procesamiento Digital de
la Señal

TRABAJO PRACTICO INTEGRADOR EJE N°1

TRABAJO PRÁCTICO INTEGRADOR - EJE I

Fundamentos y Procesamiento Digital de la Señal

Materia: Procesamiento Digital de Señales de Audio

Duración estimada: 6-8 horas

Modalidad: Grupos

Entrega: Notebook de Google Colab + Informe en PDF

CONSIGNAS GENERALES

Objetivos del Trabajo Práctico:

Al finalizar este TP, deberás ser capaz de:

1. Aplicar correctamente los conceptos de muestreo, cuantización y detección de aliasing
2. Preparar un dataset de audio crudo aplicando técnicas de limpieza y normalización
3. Realizar un análisis exploratorio completo (EDA) utilizando la Transformada de Fourier
4. Extraer conclusiones fundamentadas sobre las características de las señales analizadas
5. Documentar el proceso de manera clara y reproducible

Escenario:

Trabajarás con un **dataset de comandos de voz** que contiene grabaciones de las palabras "sí", "no", "izquierda" y "derecha". Las grabaciones fueron realizadas en condiciones no controladas (diferentes micrófonos, ambientes ruidosos, duraciones variables).

Tu misión es:

- **Diagnosticar** la calidad de las grabaciones
- **Preparar** los datos para análisis posterior
- **Analizar** las características de cada clase
- **Proponer** recomendaciones para mejorar el dataset

PARTE 1: FUNDAMENTOS TEÓRICOS (20 puntos)

Pregunta 1.1: Muestreo y Teorema de Nyquist (5 puntos)

- a) Explicá con tus palabras qué es el muestreo y por qué es necesario digitalizar una señal de audio.
- b) Una señal de voz tiene componentes de frecuencia hasta 4000 Hz.
- ¿Cuál es la frecuencia de muestreo mínima teórica necesaria?
 - Si muestreemos a 6000 Hz, ¿qué problema podríamos tener?
 - ¿Por qué el audio de teléfono usa 8000 Hz?
- c) Investigá: ¿por qué el audio de CD usa 44100 Hz y no 40000 Hz exactamente?

Pregunta 1.2: Cuantización y Resolución (5 puntos)

- a) Compará dos grabaciones de la misma señal:

- Grabación A: 8 bits de resolución
- Grabación B: 16 bits de resolución

Explicá:

- ¿Cuántos niveles de amplitud tiene cada una?
 - ¿Cuál tendrá mejor calidad y por qué?
 - ¿Qué desventaja tiene la grabación B?
- b) Si una señal tiene un rango dinámico de 96 dB, ¿cuántos bits mínimos se necesitan para representarla sin perder información? (Pista: cada bit aporta ~6 dB)

Pregunta 1.3: Aliasing (5 puntos)

- a) Describí el fenómeno de aliasing con un ejemplo cotidiano (puede ser el de la rueda de carroza en películas u otro que conozcas).
- b) Matemáticamente:
- Señal original: 7000 Hz
 - Frecuencia de muestreo: 8000 Hz
 - ¿Qué frecuencia "falsa" aparecerá después del muestreo?
 - Mostrá el cálculo

c) ¿Cómo se puede prevenir el aliasing? Menciona al menos dos estrategias.

Pregunta 1.4: Dominio del Tiempo vs. Frecuencia (5 puntos)

a) Completá el cuadro comparativo:

Característica	Dominio del Tiempo	Dominio de la Frecuencia
¿Qué muestra?		
Herramienta principal		FFT
Ejemplo de uso	Detectar silencios	

b) Explicá qué es la Transformada de Fourier y por qué es importante en el análisis de audio.

c) Si ves un espectro con un pico muy alto en 440 Hz y picos más bajos en 880 Hz y 1320 Hz, ¿qué podés concluir sobre la señal?

PARTE 2: APLICACIÓN PRÁCTICA CON PYTHON (60 puntos)

Configuración Inicial

Python

```
# 📦 Instalación de librerías (ejecutar una vez)
!pip install librosa numpy matplotlib scipy pandas seaborn

# 📖 Importación de módulos
import numpy as np
import matplotlib.pyplot as plt
import librosa
import librosa.display
import pandas as pd
import seaborn as sns
from scipy import signal
from IPython.display import Audio, display

# ⚙️ Configuración
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['font.size'] = 11
np.random.seed(42) # Para reproducibilidad

print("✅ Entorno configurado correctamente")
```

Ejercicio 2.1: Generación y Análisis de Señales Sintéticas (15 puntos)

Consigna: Generarás tres señales diferentes para estudiar los efectos del muestreo y cuantización.

Python

```
# Tu código aquí
# Completar esta sección

# a) Generar una señal "analógica" de referencia (muestreada a 100000 Hz)
#   Duración: 0.1 segundos
#   Frecuencia: 1000 Hz
#   Amplitud: 1.0

# b) Generar una versión muestreada de esta señal a diferentes tasas:
#   - fs1 = 2500 Hz (muestreo adecuado)
#   - fs2 = 1500 Hz (cercano al límite de Nyquist)
#   - fs3 = 800 Hz (submuestreo - aliasing)

# c) Para cada versión muestreada:
#   - Graficar la señal original (en gris claro) y la muestreada (con stem)
#   - Calcular y mostrar la frecuencia de Nyquist
#   - Indicar si hay aliasing o no
#   - Calcular el error RMS entre la original y la reconstruida

# d) Aplicar cuantización a la señal fs1:
#   - Versión 1: 4 bits
#   - Versión 2: 8 bits
#   - Versión 3: 16 bits
#   - Graficar las tres versiones superpuestas
#   - Calcular el SNR (relación señal-ruido) de cuantización para cada una
```

Entregables:

- 3 gráficos comparativos (muestreo)
- 1 gráfico comparativo (cuantización)
- Tabla con errores RMS y SNR
- Conclusiones sobre los efectos observados

Ejercicio 2.2: Creación de Dataset Sintético (10 puntos)

Consigna: Crear un dataset simulado de comandos de voz con características controladas.

Python

```
# Tu código aquí

# Generar 40 muestras de audio (10 por clase)
# Clases: "grave", "medio", "agudo", "ruido"

# Especificaciones:
# - Sample rate: 22050 Hz
# - Duración base: 2.0 segundos
# - Cada clase debe tener características distintivas:
#   * Grave: frecuencias principales 200-400 Hz
#   * Medio: frecuencias principales 600-1000 Hz
#   * Agudo: frecuencias principales 1500-2500 Hz
#   * Ruido: ruido blanco filtrado

# Agregar variabilidad:
# - Ruido de fondo aleatorio (SNR entre 10-20 dB)
# - Duración variable (1.5 a 2.5 segundos)
# - Amplitud variable (0.5 a 1.0)

# Guardar el dataset en una lista de diccionarios:
# [{
#   'audio': array de muestras,
#   'sr': sample_rate,
#   'clase': etiqueta,
#   'duracion': duración_real,
#   'amplitud_max': valor_maximo
# }, ...]

print(f"Dataset generado: {len(dataset)} muestras")
```

Ejercicio 2.3: Preparación de Datos Crudos (15 puntos)**Consigna:** Aplicar el pipeline completo de preprocessing al dataset generado.

Python

```

# Tu código aquí

# Implementar una función de preprocessing que:
def preparar_audio(audio, sr, duracion_objetivo=2.0, top_db=30):
    """
    Pipeline completo de preparación:
    1. Convertir a mono si es necesario
    2. Recortar silencios (trimming)
    3. Normalizar amplitud (peak normalization)
    4. Remuestrear a 22050 Hz si es necesario
    5. Ajustar duración (pad o crop)

    Retorna: audio_procesado, sr, metadatos
    """
    # Completar implementación

    pass

# Aplicar la función a todo el dataset
dataset_procesado = []

for i, muestra in enumerate(dataset):
    # Procesar cada audio
    # Guardar metadatos antes y después del procesamiento
    # Calcular:
    #   - Duración original vs final
    #   - Amplitud máxima original vs final
    #   - Porcentaje de silencios recortados

    pass

# Crear DataFrame comparativo
df_metadatos = pd.DataFrame(metadatos_lista)

# Visualizar:
# - Histograma de duraciones (antes y después)
# - Comparación de amplitudes máximas
# - Porcentaje de reducción por trimming

print("✓ Dataset procesado y guardado")

```


Entregables:

- Función preparar_audio() completamente implementada
- DataFrame con metadatos comparativos
- 3 gráficos de visualización
- Análisis de cuánto se redujo el dataset en promedio

Ejercicio 2.4: Análisis Exploratorio - FFT y Espectros (10 puntos)

Consigna: Analizar el contenido frecuencial de cada clase del dataset.

python

```
# Tu código aquí

# a) Seleccionar un ejemplo representativo de cada clase
# (el que tenga RMS más cercano a la media de su clase)

# b) Para cada ejemplo:
# - Calcular FFT (usar n_fft=2048)
# - Graficar el espectro de magnitudes (0-5000 Hz)
# - Identificar y marcar los 3 picos principales
# - Calcular:
#   * Frecuencia fundamental (pico más bajo significativo)
#   * Frecuencia del pico más alto
#   * Ancho de banda a -20 dB del pico principal

# c) Comparar espectros promedio por clase:
# - Calcular el espectro promedio de todas las muestras de cada clase
# - Graficar los 4 espectros promedio superpuestos
# - Identificar diferencias características entre clases

# d) Responder:
# - ¿Se pueden distinguir las clases solo mirando el espectro?
# - ¿Qué rangos de frecuencia son más discriminativos?
# - ¿Hay solapamiento entre clases?
```

Entregables:

- 4 gráficos de espectros individuales (uno por clase)
- 1 gráfico de espectros promedio comparativos
- Tabla con frecuencias características por clase
- Conclusiones sobre discriminabilidad

Ejercicio 2.5: Extracción de Features y EDA Estadístico (10 puntos)**Consigna:** Extraer features y realizar análisis estadístico del dataset.

Python

```

# Tu código aquí

# a) Extraer las siguientes features para cada muestra:
features_lista = []

for muestra in dataset_procesado:
    audio = muestra['audio']
    sr = muestra['sr']

    features = {
        'clase': muestra['clase'],
        'duracion': len(audio) / sr,
        'rms': np.sqrt(np.mean(audio**2)),
        'zcr': librosa.feature.zero_crossing_rate(audio)[0].mean(),
        'spectral_centroid': librosa.feature.spectral_centroid(y=audio,
sr=sr)[0].mean(),
        'spectral_bandwidth': librosa.feature.spectral_bandwidth(y=audio,
sr=sr)[0].mean(),
        'spectral_rolloff': librosa.feature.spectral_rolloff(y=audio, sr=sr)[0].mean(),
        'spectral_flatness': librosa.feature.spectral_flatness(y=audio)[0].mean()
    }

    features_lista.append(features)

# Convertir a DataFrame
df_features = pd.DataFrame(features_lista)

# b) Estadísticos descriptivos:
# - Calcular media, std, min, max por clase para cada feature
# - Mostrar tabla resumen

# c) Visualizaciones:
# - Boxplot de cada feature comparando las 4 clases
# - Pairplot de las 3 features más importantes
# - Heatmap de correlación de todas las features

# d) Análisis de correlación:
# - Identificar pares de features con |correlación| > 0.7
# - Discutir redundancia

```

```
# e) PCA:  
# - Aplicar PCA con 2 componentes principales  
# - Graficar proyección 2D coloreando por clase  
# - Calcular varianza explicada  
# - Evaluar si las clases están separables
```

Entregables:

- DataFrame completo con features
- Tabla de estadísticos descriptivos
- 4 boxplots (uno por feature principal)
- Heatmap de correlación
- Gráfico PCA
- Conclusiones sobre separabilidad de clases

PARTE 3: ANÁLISIS E INTERPRETACIÓN (20 puntos)

Ejercicio 3.1: Informe Técnico (15 puntos)

Redactar un informe de máximo 3 páginas que incluya:

1. **Introducción** (2 puntos)
 - Objetivo del trabajo
 - Descripción del dataset
 - Metodología aplicada
2. **Resultados del Preprocessing** (4 puntos)
 - Problemas detectados en los datos crudos
 - Transformaciones aplicadas y justificación
 - Impacto cuantitativo (reducción de duración, normalización, etc.)
3. **Análisis Exploratorio** (6 puntos)
 - Características distintivas de cada clase
 - Features más discriminativas identificadas
 - Correlaciones importantes detectadas
 - Visualización de la separabilidad (PCA)
4. **Conclusiones y Recomendaciones** (3 puntos)
 - ¿Es adecuado el dataset para clasificación?
 - ¿Qué mejoras propondrías?
 - ¿Qué features recomendarías usar para un modelo de ML?

Ejercicio 3.2: Reflexión Crítica (5 puntos)

Responder brevemente (máximo 1 carilla):

- a) Si tuvieras que explicar a alguien sin conocimientos técnicos qué es el análisis espectral de audio, ¿cómo lo harías? Usá una analogía.
- b) Durante el TP, ¿qué concepto te resultó más difícil de entender? ¿Cómo lo superaste?

- c) Imaginá que trabajás en una empresa que desarrolla un asistente de voz. Basándote en lo aprendido, ¿qué recomendaciones le darías al equipo de recolección de datos?
- d) ¿Cómo aplicarías estos conocimientos en tu proyecto final o en un contexto laboral real?

ENTREGA

Formato:

1. **Notebook de Google Colab** (.ipynb)
 - Nombre: TP1_Eje1_Apellido_Nombre.ipynb
 - Debe ejecutar sin errores de principio a fin
 - Incluir celdas de markdown explicando cada sección
 - Código comentado y prolijo
2. **Informe en PDF**
 - Nombre: TP1_Informe_Apellido_Nombre.pdf
 - Incluir gráficos generados
 - Respuestas a preguntas teóricas
 - Conclusiones personales
3. **Link al Colab** (en el informe)
 - Asegurarse de que sea accesible (compartido públicamente o con el docente)
4. **Link al Repositorio GITHUB**
 - Asegurarse de que sea accesible (compartido públicamente o con el docente)

Fecha de entrega:

- **Fecha límite:** [Indicar fecha]
- **Entrega tardía:** -1 punto por día de retraso

Criterios de Evaluación:

Criterio	Ponderación
Correcta aplicación de conceptos teóricos	20%
Implementación funcional del código	30%
Calidad del análisis y visualizaciones	25%
Claridad en la documentación	15%
Reflexión crítica y conclusiones	10%
Total	100%

RECURSOS Y BIBLIOGRAFÍA

Obligatoria:

- Material teórico de los Temas 1, 2 y 3 del Eje I
- Notebooks de los Laboratorios 1, 2 y 3
- Documentación de [Librosa](#)

Complementaria:

- [Speech and Language Processing - Jurafsky](#) - Capítulos sobre procesamiento de señal
- [NLTK Book](#) - Sección de features
- Tutoriales de FFT y espectrogramas en YouTube

Foros de Consulta:

- Stack Overflow (etiquetas: python, audio, librosa, fft)
- Foro de la materia en Campus Virtual

IMPORTANTE

- **Trabajo original:** Se permite consultar bibliografía y foros, pero el código y análisis deben ser propios.
- **Plagio:** Copiar código o informes de compañeros resultará en calificación cero para todos los involucrados.
- **Consultas:** Ante dudas, consultar al docente antes de la fecha de entrega.
- **Backup:** Guardar copias de seguridad del trabajo. Google Colab puede desconectarse.

¡Éxitos en el trabajo!